Figures for the report

Pipelines and code structures

Five figures: the shared experimental pipeline, how each of the three schemes builds its codeword, and the block-circulant decoder whose radius sets BC's erasure tolerance. Everything here is either a pipeline the report has to describe anyway, or the structure behind a measured result. Earlier drafts also carried worked numeric examples and algorithm flowcharts; those were teaching aids rather than results and have been dropped.

Figures are drawn from this project's own code (`1D_RS_Galois.py`, `2D_RS_Galois.py`, `BCcodeEthereum.py`). The BC figures follow the published construction — Sasidharan, Viterbo & Hoang Dau, *"Block Circulant Codes with Application to Decentralized Systems,"* IEEE Trans. Commun. 73(10), 2025 (arXiv:2406.12160) — since that is what the implementation is meant to realise.

Experimental pipeline

Every number in the benchmark report comes from running this loop once per trial. The blob is fixed and the erasure rate is swept; at each rate the codeword is corrupted at randomly chosen positions and handed to the decoder. A trial counts as a success only if the bytes that come out the far end are identical to the ones that went in.

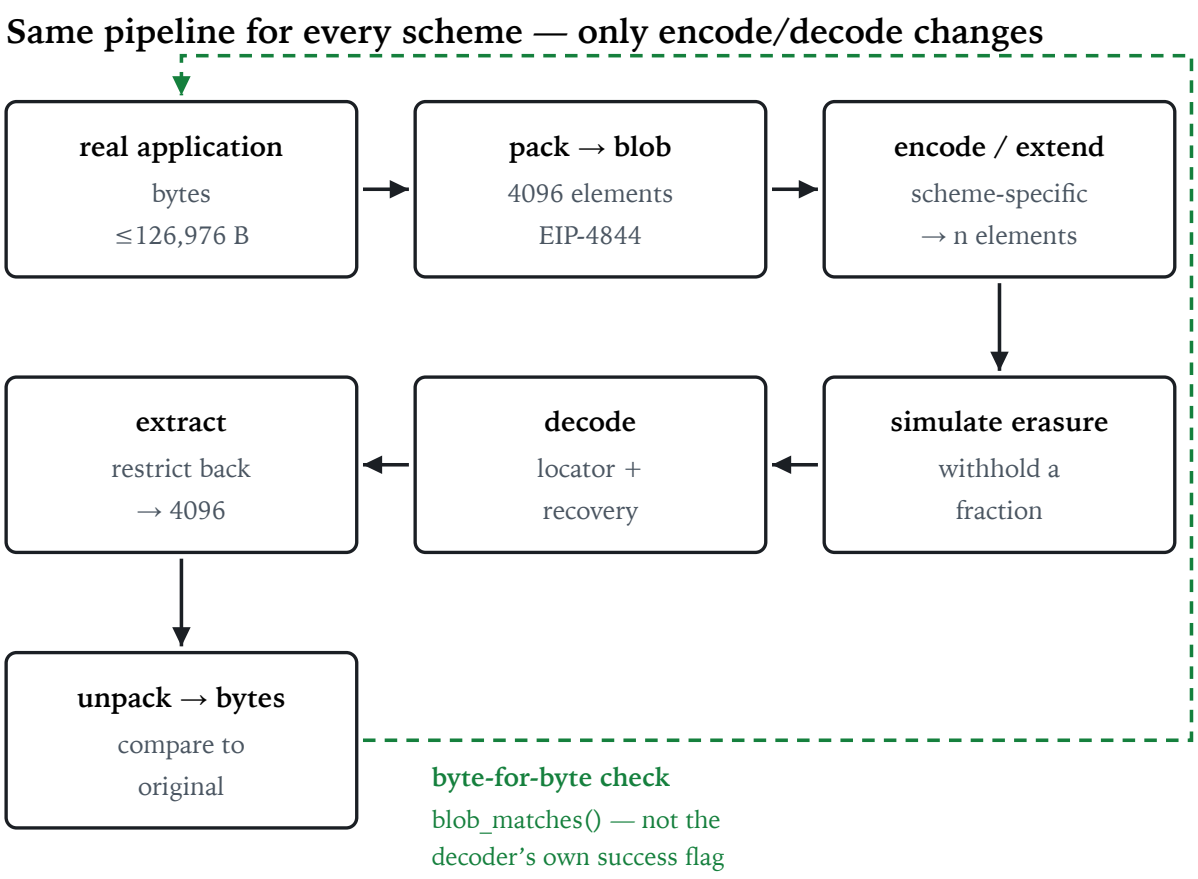


Fig. 1. The measurement pipeline. Real application bytes are packed into one EIP-4844 blob (4096 field elements, 31 usable bytes each), the scheme under test expands that blob into a longer codeword, a fraction of the codeword is withheld to stand in for samples peers failed to serve, the decoder tries to rebuild it, and the result is unpacked back to bytes. Only the two shaded-in-meaning stages — *encode/extend* and *decode* — differ between the three schemes; the input, the erasure model and the test of success are byte-identical for all of them, which is what makes a difference in the results attributable to the code rather than to the setup. The dashed return path is the success test itself: the recovered bytes must equal the original bytes exactly. It deliberately does not ask the decoder whether it succeeded — one of the three decoders was found reporting success on data it had silently corrupted, and only this comparison caught it.

How each scheme builds its codeword

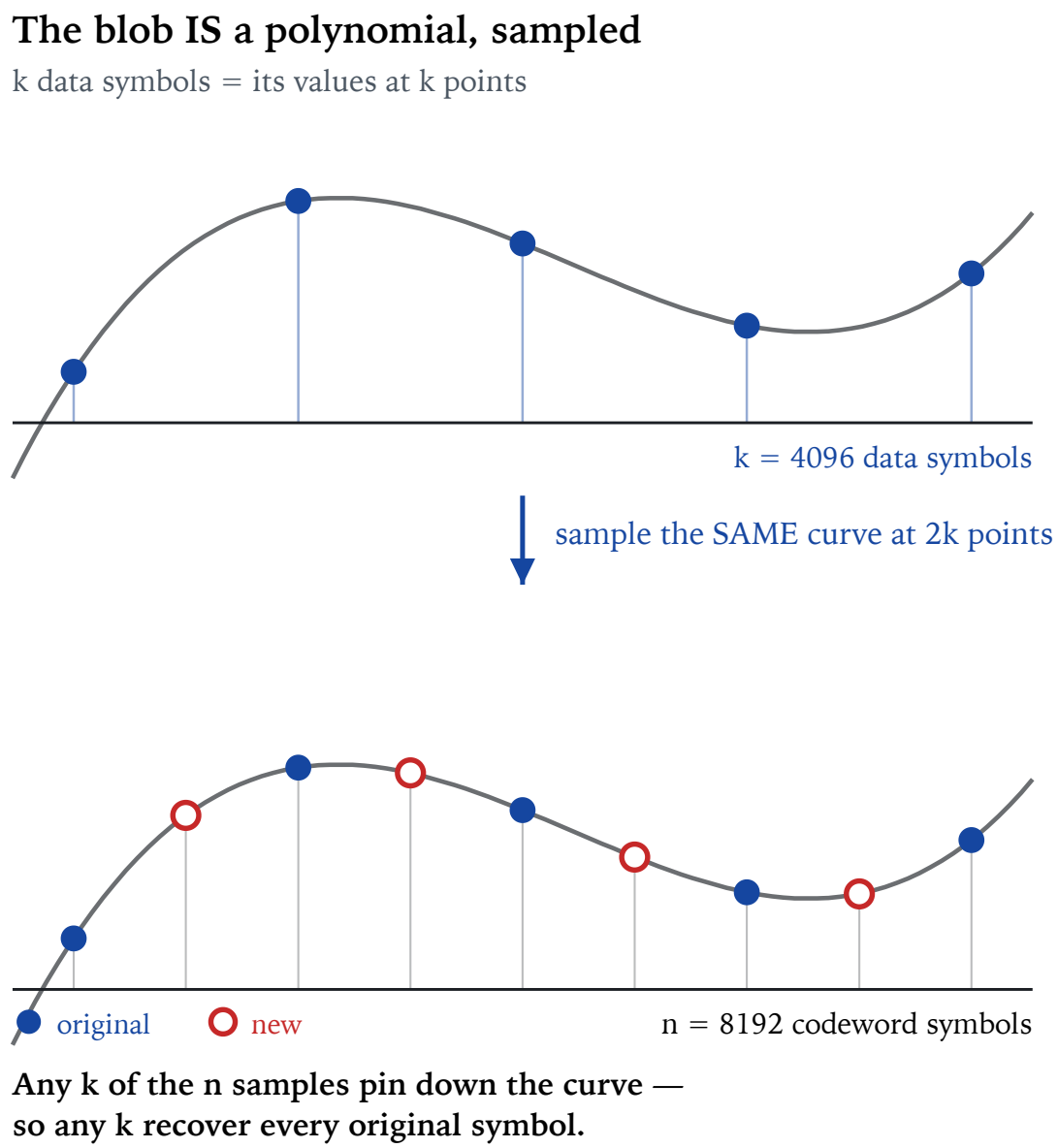


Fig. 2. 1D-RS. The blob's k field elements are not raw bytes to be copied; they are the *values of a single polynomial* sampled at k points. Encoding finds that polynomial and samples it at n = 2k points instead. Because a degree-<k polynomial is pinned down by any k of its values, any k surviving symbols reconstruct the curve and hence every original symbol. 2× overhead.

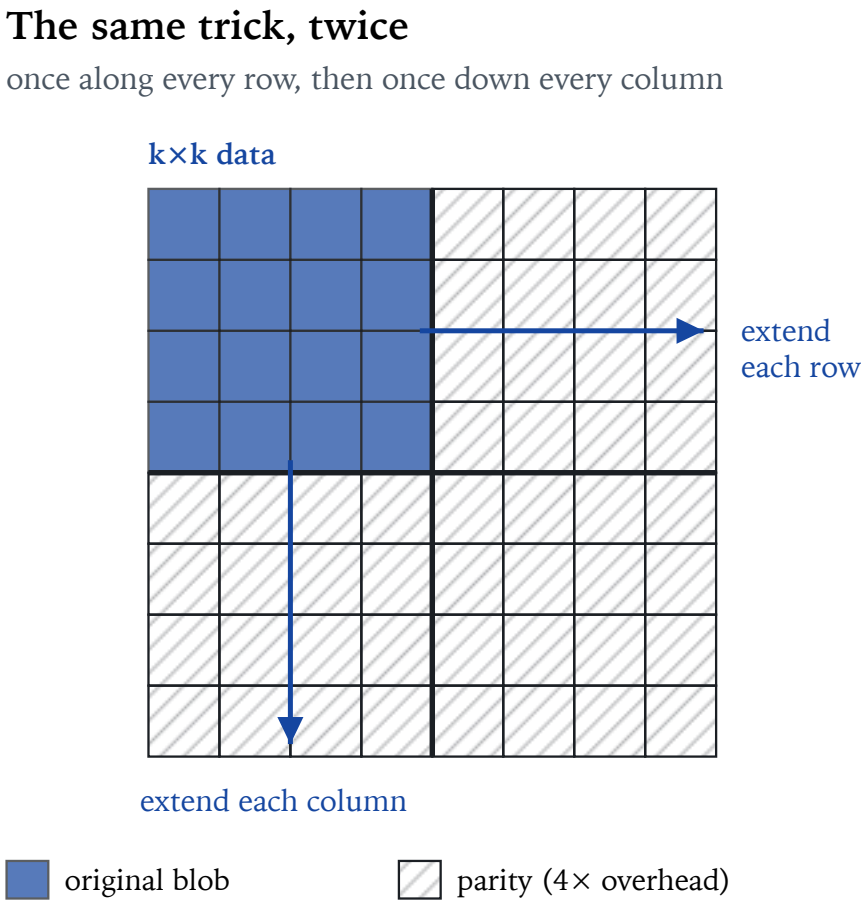


Fig. 3. 2D-RS. The blob is reshaped into a k×k grid and the 1D-RS step is applied twice: once to extend every row, then once to extend every column of the result. The outcome is an n×n grid at 4× overhead in which every row and every column is independently a 1D-RS codeword — so a lost cell can be repaired from whichever of its two lines has enough survivors, which is what lets the iterative decoder cascade.

Block-circulant structure

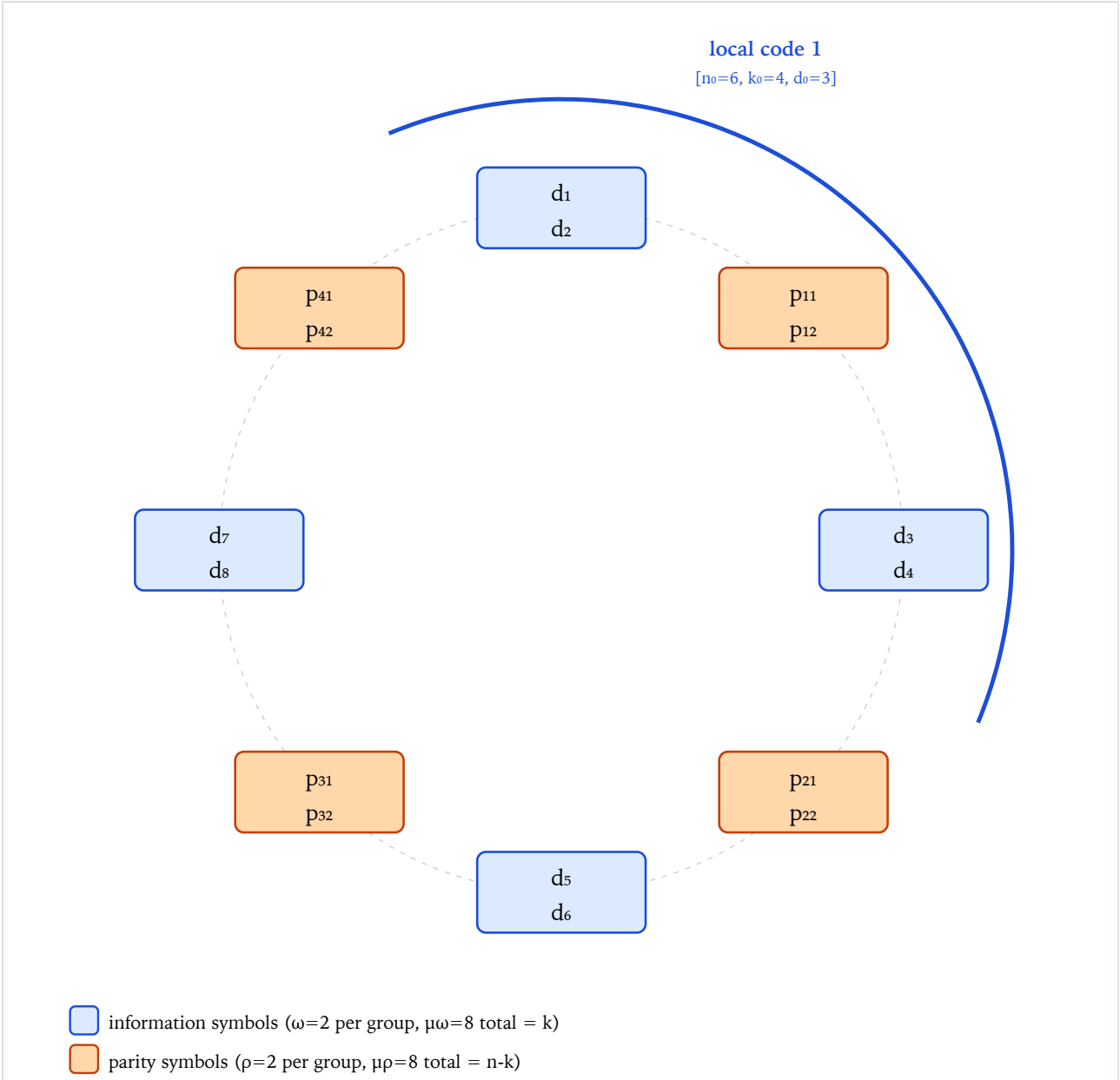


Fig. 4. Block-circulant layout (drawn at μ=4 for legibility; the benchmark runs μ=2048). Information blocks alternate with parity blocks around a ring, and each local code covers its own parity plus the information of λ consecutive blocks. 2× overhead, the same as 1D-RS, but the parity is local rather than global.

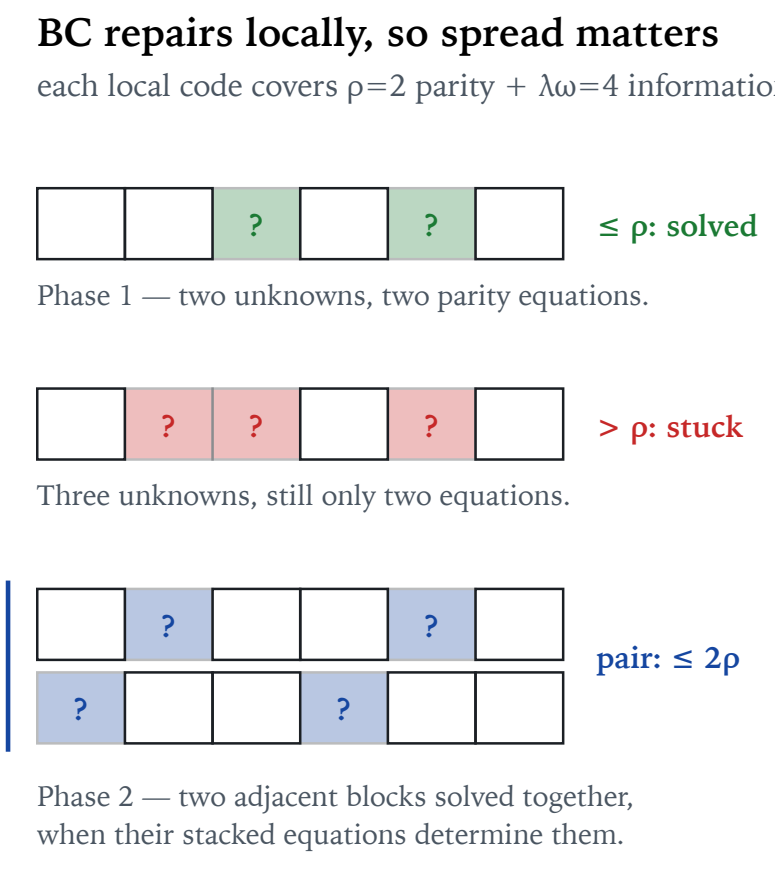


Fig. 5. Why BC's tolerance is limited. Repair is local: a block holding at most ρ erasures is solved by its own two parity equations; one holding more is stuck; an adjacent pair holding at most 2ρ between them can sometimes be solved jointly. The guaranteed correction is only $d-1 = 4$ erasures across the entire 8192-symbol codeword, so beyond that BC depends on erasures staying thinly spread across the $\mu=2048$ local blocks — which random erasure stops providing well below the rates the Reed-Solomon variants reach.